

Machine Learning for Performance and Power Modeling/Prediction

Lizy K. John
University of Texas at Austin

Simulation Challenges

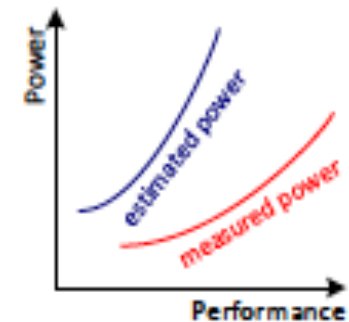
- **Simulation Based Performance Models**
 - eg: SimOS, SIMICS, GEM5, SimpleScalar
- **Power modeling**
 - eg: McPAT, CACTI
- **Full system Simulation is prohibitively slow**
- **Simulation errors**
- **Large Gap between what's evaluated pre-silicon and what's run post-silicon**

Three Examples for Using Machine Learning in Performance/Power Modeling and Prediction

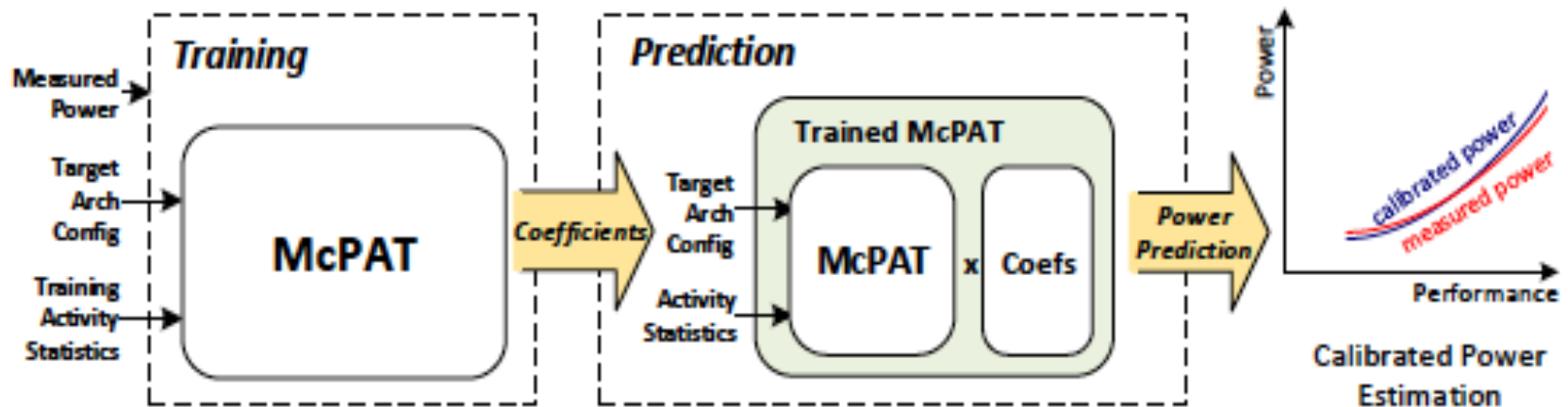
- **Calibration of Power Models using Machine Learning**
- **Cross-Platform Performance/Power Prediction using Machine Learning**
- **Stressmarks and Power Viruses using Machine Learning**

Example 1

Machine Learning for Model Calibration ISLPED 2015



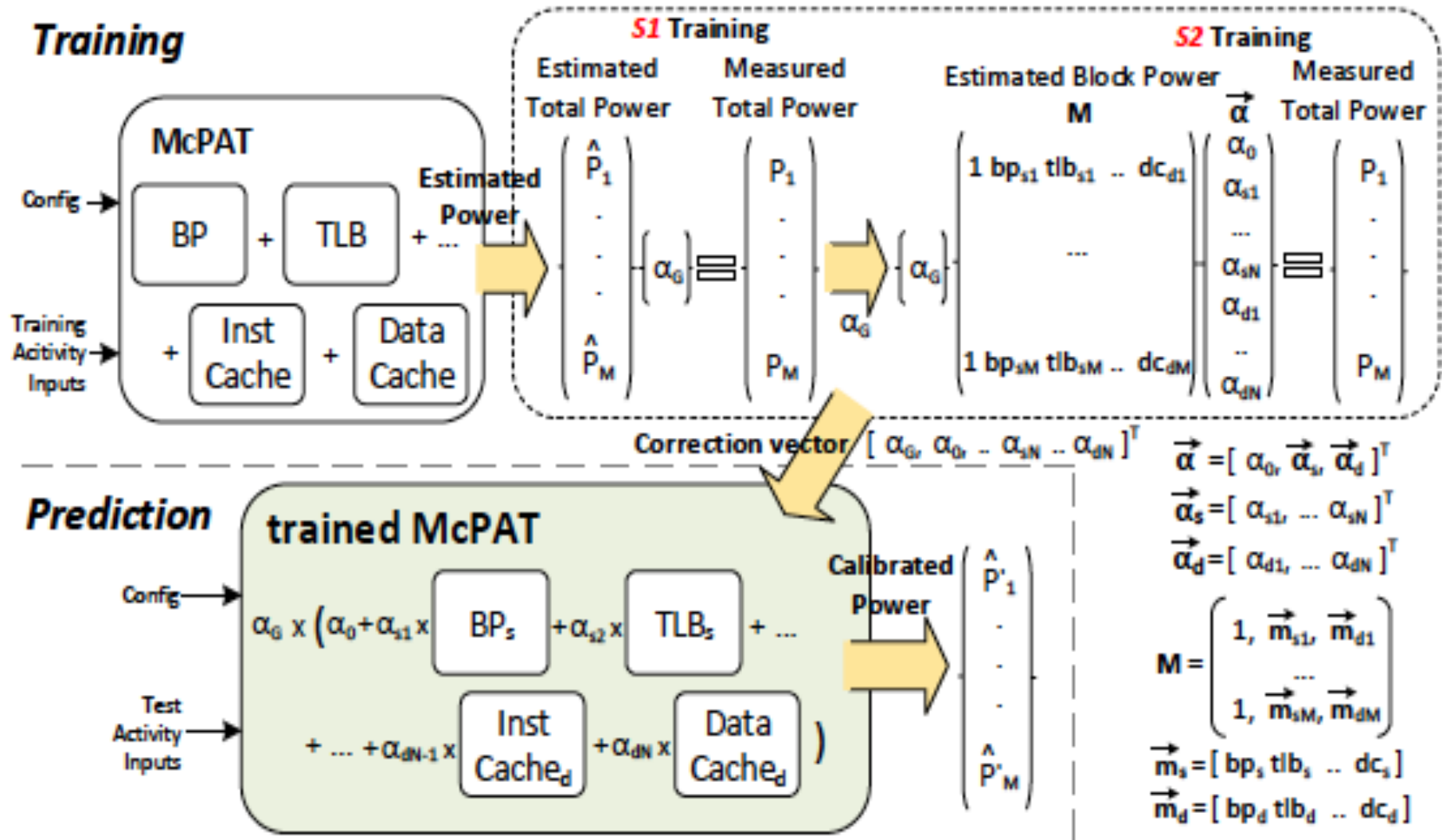
(a) McPAT



(b) PowerTrain

Example 1

Machine Learning for Model Calibration



Correction Factors – Additive and Multiplicative

$$\hat{p}_m = \alpha_G \times (\alpha_0 + \vec{\mathbf{m}}_s \times \vec{\alpha}_s + \vec{\mathbf{m}}_d \times \vec{\alpha}_d)$$

Non-negative Least Square Error Solver

$$\begin{aligned} & \text{minimize } \|\mathbf{M}\vec{\alpha} - \vec{\mathbf{P}}\|^2 + \lambda \|\vec{\alpha}\|, \\ & \text{subject to } \alpha \geq 0, \quad \lambda = 0.01 \end{aligned}$$

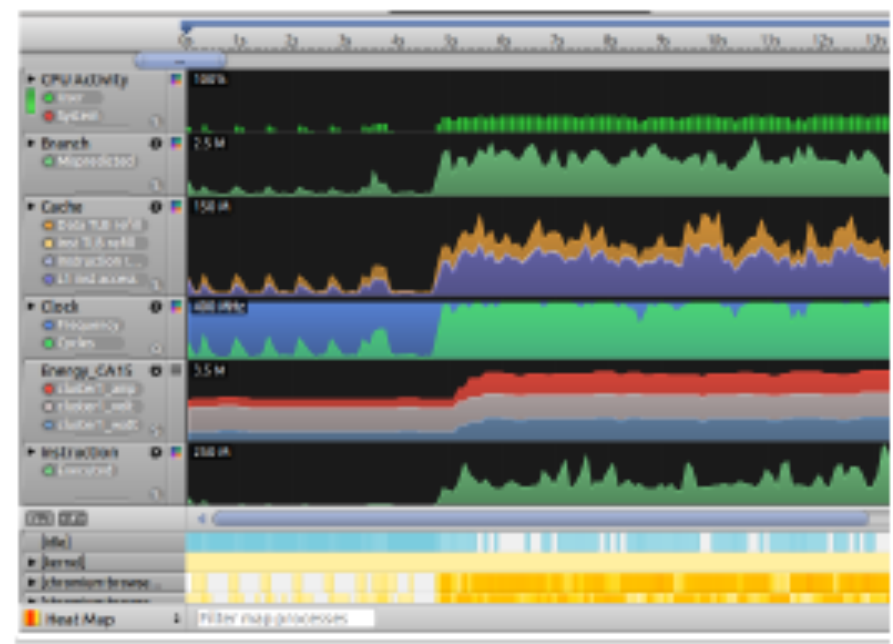
$$\hat{p}_m = \alpha_G \times (\alpha_0 + \vec{\mathbf{m}}_s \times \vec{\alpha}_s + \vec{\mathbf{m}}_d \times \vec{\alpha}_d)$$

Example 1

Training Process



(a) Odroid Board



(b) DS-5

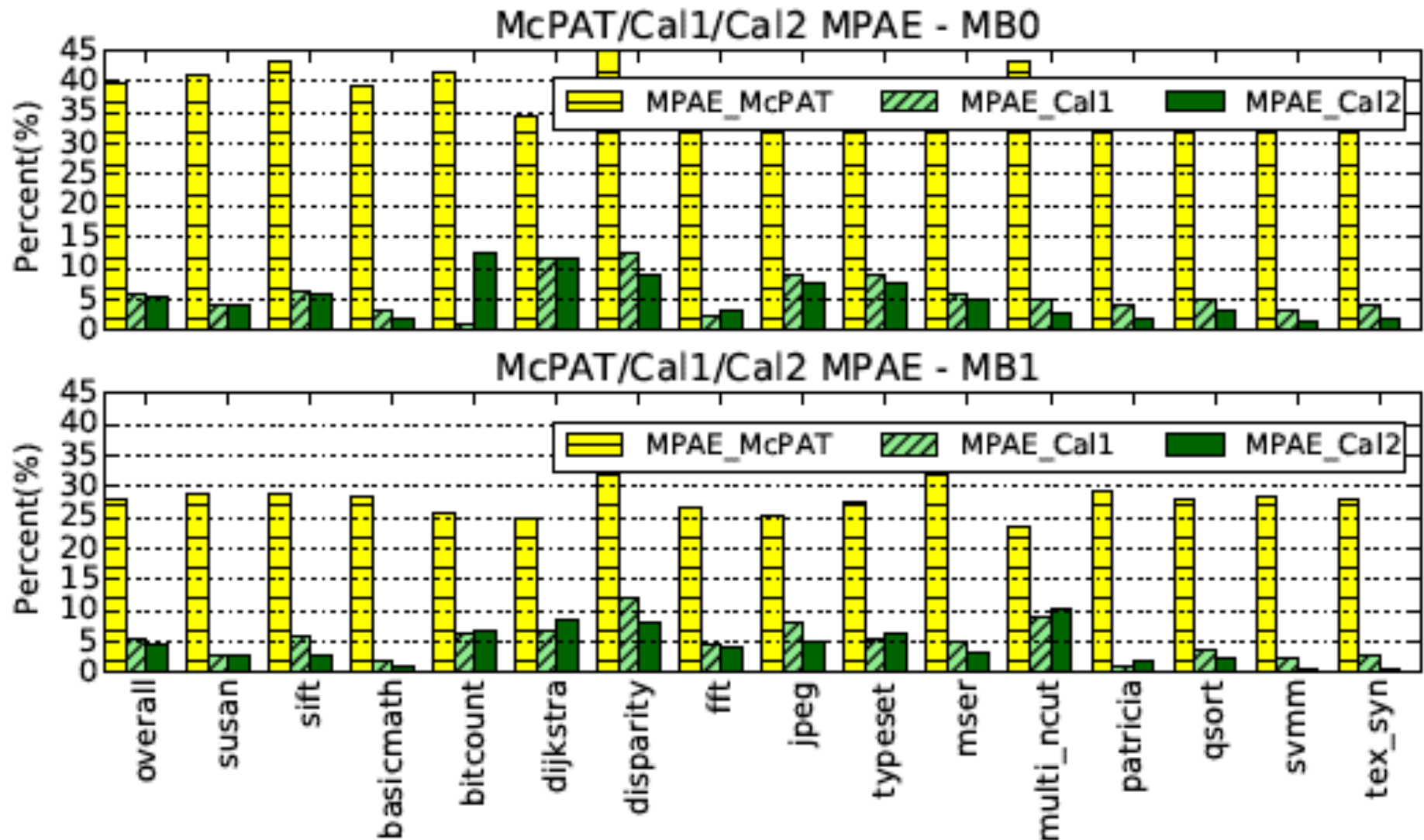
Example 1

Training Process

Performance Counter Event	Description
CPU_CYCLES	Cycle
INST_RETIRED	Inst architecturally executed
INST_SPEC	Inst spec exec
LD_SPEC	Inst spec exec, load
ST_SPEC	Inst spec exec, store
ASE_SPEC	Inst spec exec, Advanced SIMD
VFP_SPEC	Inst spec exec, floating-point
DP_SPEC	Inst spec exec, int data processing
BR_IMMED_SPEC	Branch spec exec, immediate branch
BR_RETURN_SPEC	Branch spec exec, procedure return
BR_INDIRECT_SPEC	Branch spec exec, indirect branch
BR_MIS_PRED	Mis- or not predicted branch spec
L1I_CACHE	Level 1 instruction cache access
L1I_CACHE_REFILL	Level 1 instruction cache refill
L1I_TLB_REFILL	Level 1 instruction TLB refill
L1D_CACHE_LD	Level 1 data cache access, read
L1D_CACHE_ST	Level 1 data cache access, store
L1D_CACHE_REFILL_LD	Level 1 data cache refill, read
L1D_CACHE_REFILL_ST	Level 1 data cache refill, write
L1D_CACHE_WB	Level 1 data cache write-back
L1D_TLB_REFILL	Level 1 data TLB refill
L2D_CACHE_LD	Level 2 data cache access, read
L2D_CACHE_ST	Level 2 data cache access, write
L2D_CACHE_REFILL_LD	Level 2 data cache refill, read
L2D_CACHE_REFILL_ST	Level 2 data cache refill, write
L2D_CACHE_WB	Level 2 data cache write-back

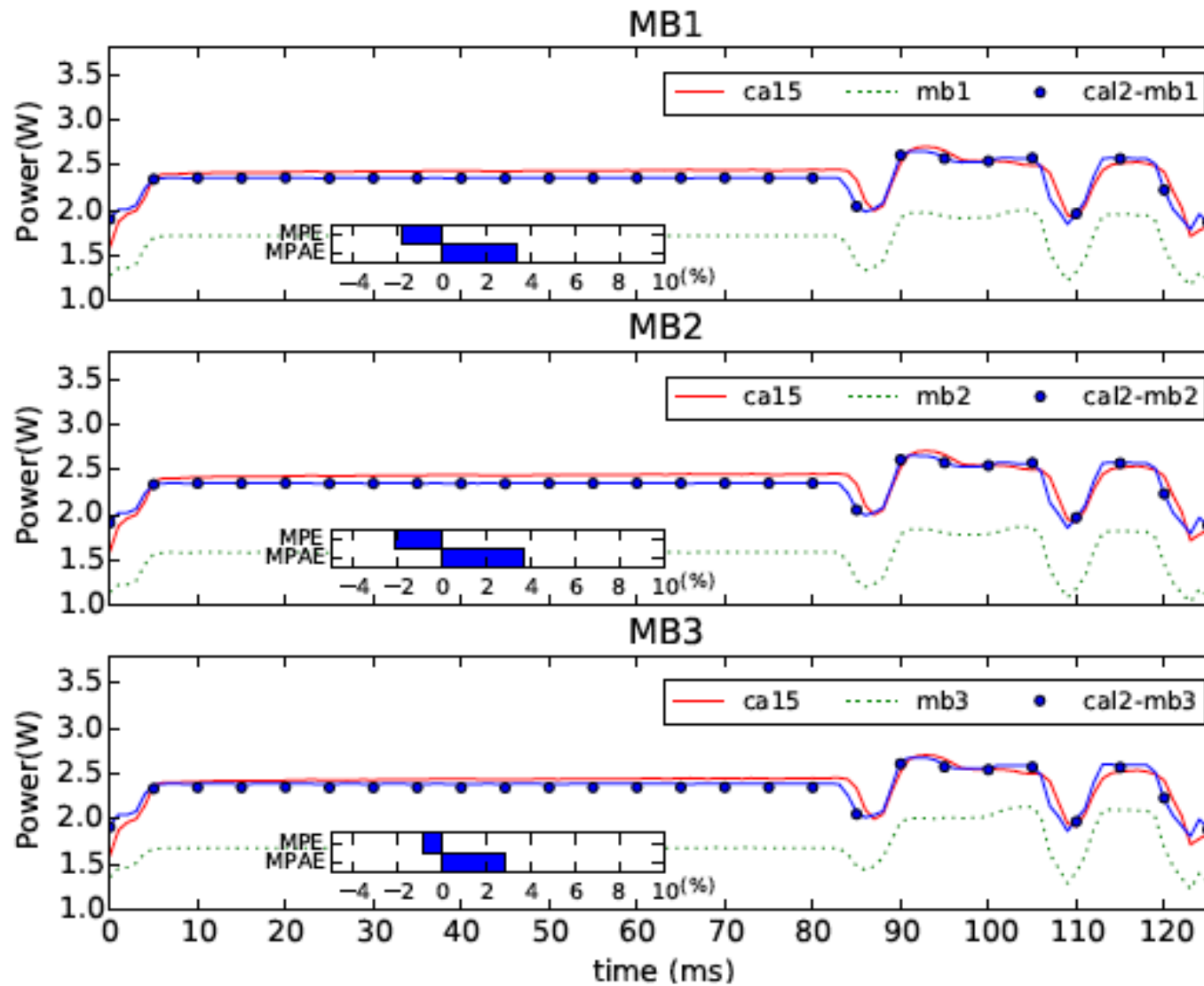
Example 1

Machine Learning for Model Calibration



Example 1

Calibrated Power



Machine Learning for Cross-Platform Prediction

Motivation:

Full System Simulation is too slow.

Analytical Models are not accurate enough.

Bridge the gap between the two using machine learning.

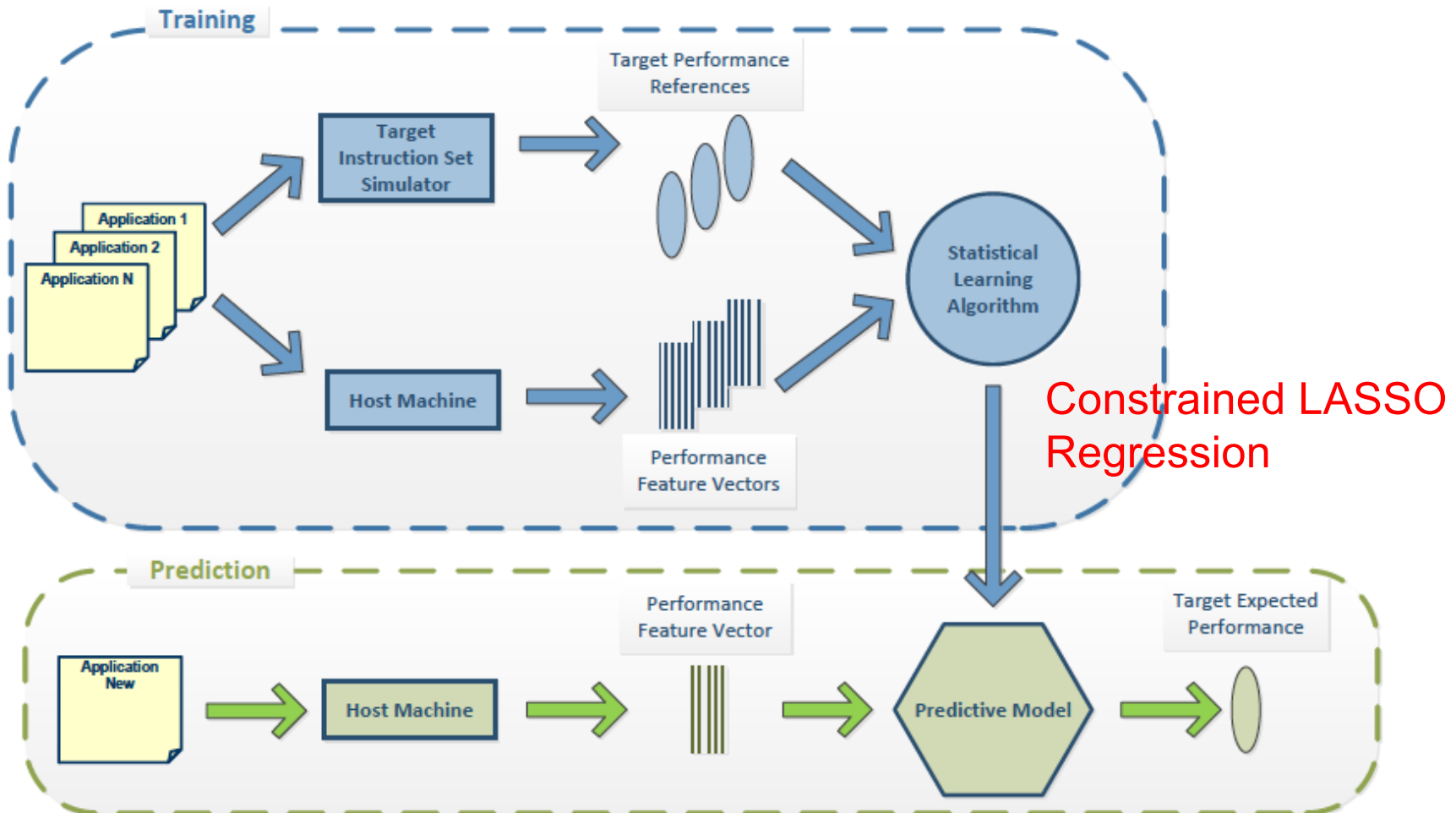
Intuition:

Performance on two platforms is correlated.

Can machine learning be used to understand that correlation?

Example 2

Machine Learning for Cross-Platform Prediction (DAC 2016, DATE 2017)



Use Cases for Cross-Platform Prediction

Slow Simulator – No time to run all benchmarks but fast previous generation or other ISA hardware available -
Run some benchmarks and use machine learning

Limited Access to New Hardware – Make some runs –
Train using them and predict power of other benchmarks

Hardware Software Co-development - If they can run code on existing hardware and predict based on the cross-platform model provided by hardware developer

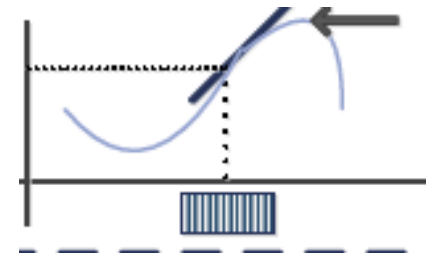
Learning Formulation

For each application i in the training set we have,

- $x_i \in R^d$: d -dimensional feature vector obtained from the host
- $y_i \in R$: reference performance on the target

We want to find a function $\mathcal{F}$ such that,

$$\mathcal{F}(x_i) \approx y_i$$



Prediction models:

- Assume locally linear $\mathcal{F}_t$ at input x_t ($\mathcal{F}_t(x_t) = w_t^T x_t$)
- Solve constrained LASSO regression around a neighborhood of x_t

Training Set – ACM Programming Contest

Application Domains	Number of Programs
Simulation	14
Enumeration	16
String Manipulation	30
Graph Algorithm	26
Dynamic Programming	21
Geometry	25
Recursion	13
Miscellaneous	12

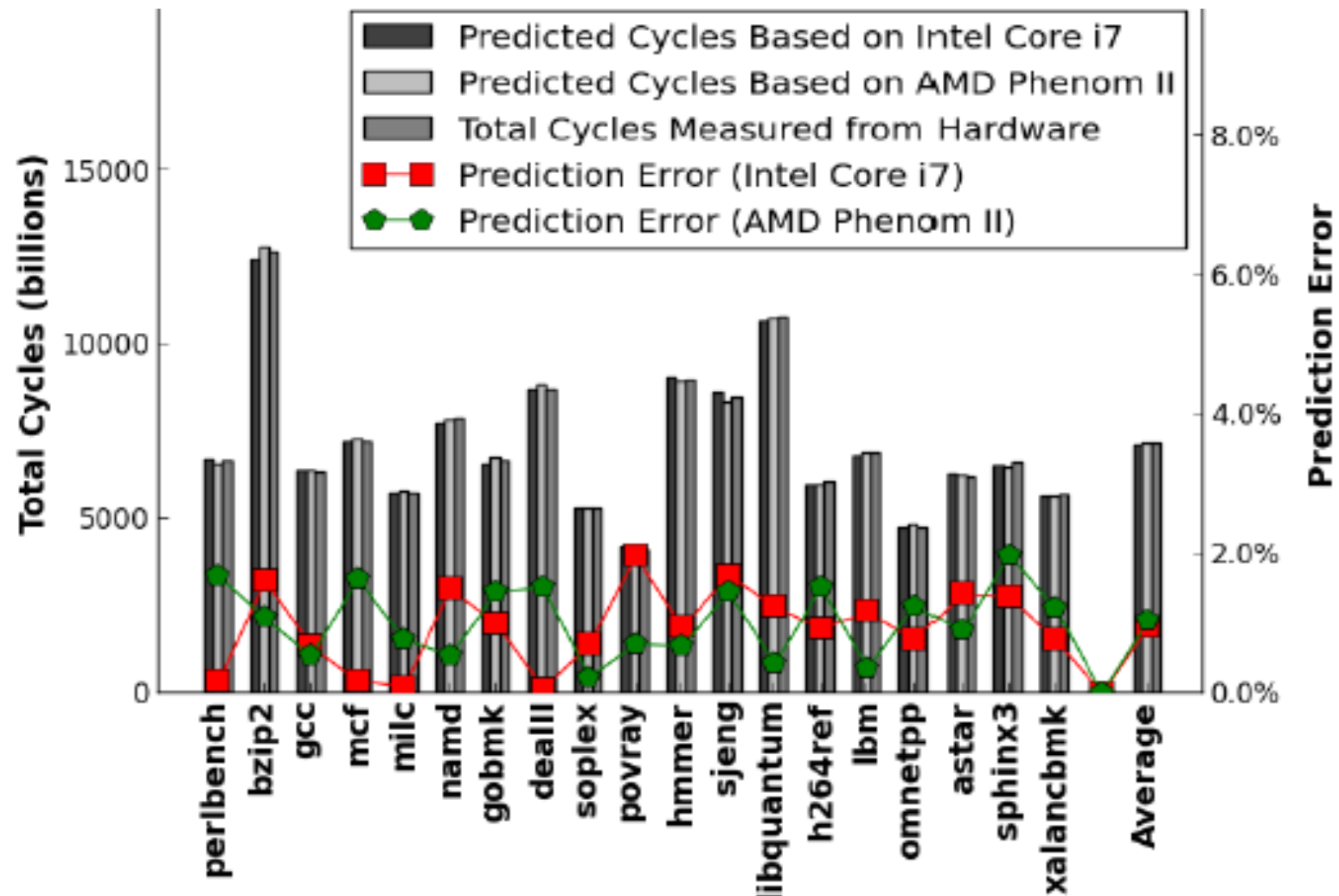
Example 2

Profiling for Training

Intel Core i7-920	AMD Phenom II X6
L1 Total Cache Misses	L1 Total Cache Misses
L2 Total Cache Misses	L2 Total Cache Misses
L3 Total Cache Misses	Branch Misses
TLB Loads	Instructions
Unconditional Branches	Cycle Stalled
Conditional Branches	Cycles
Branch Misses	L1 Total Cache Accesses
Instructions	Floating Point Operations
Cycle Stalled	
Cycles	
L1 Total Cache Accesses	
L2 Total Cache Accesses	

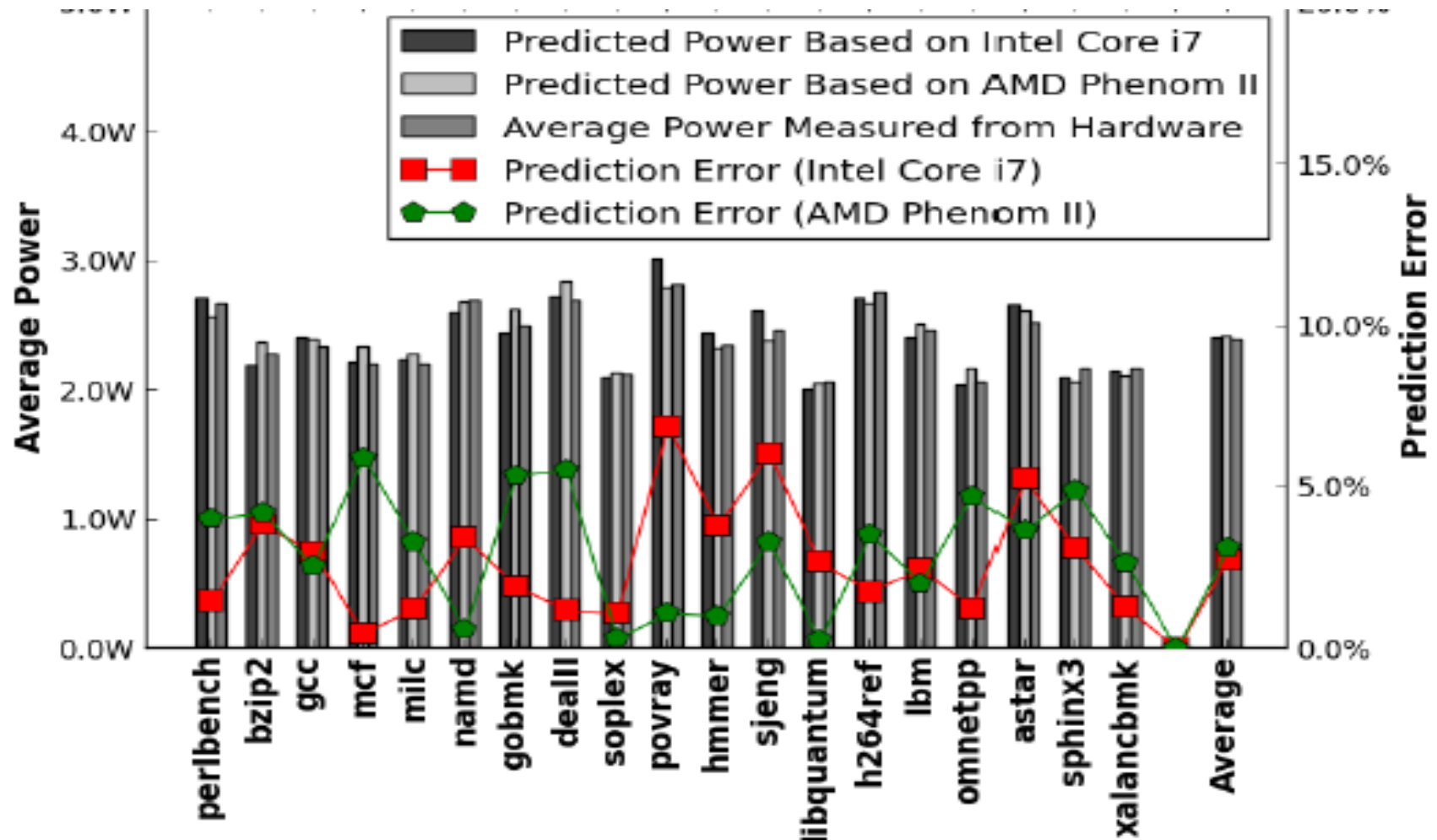
Example 2

Performance Prediction Accuracy



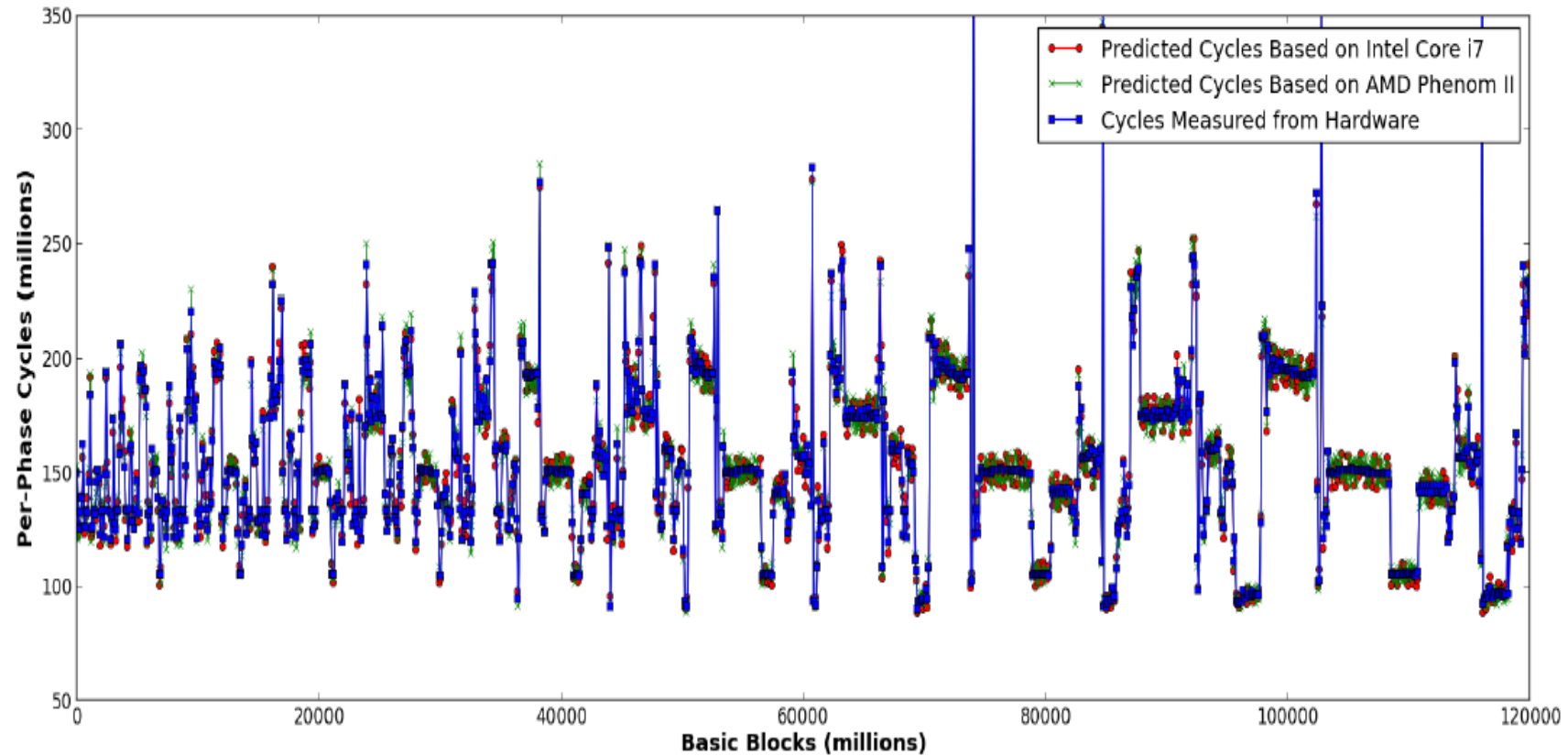
Example 2

Power Prediction - Accuracy



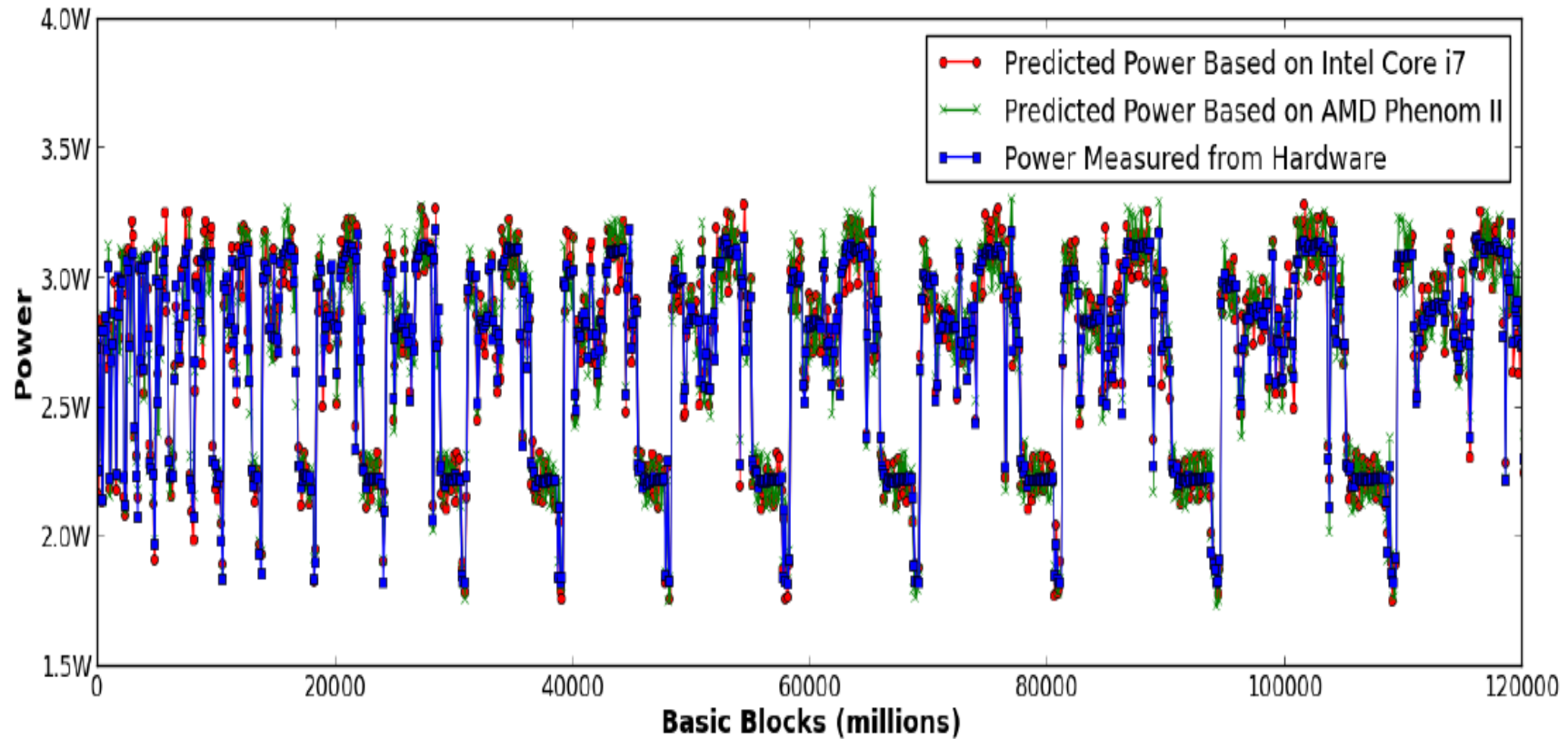
Example 2

Prediction at Fine-grain



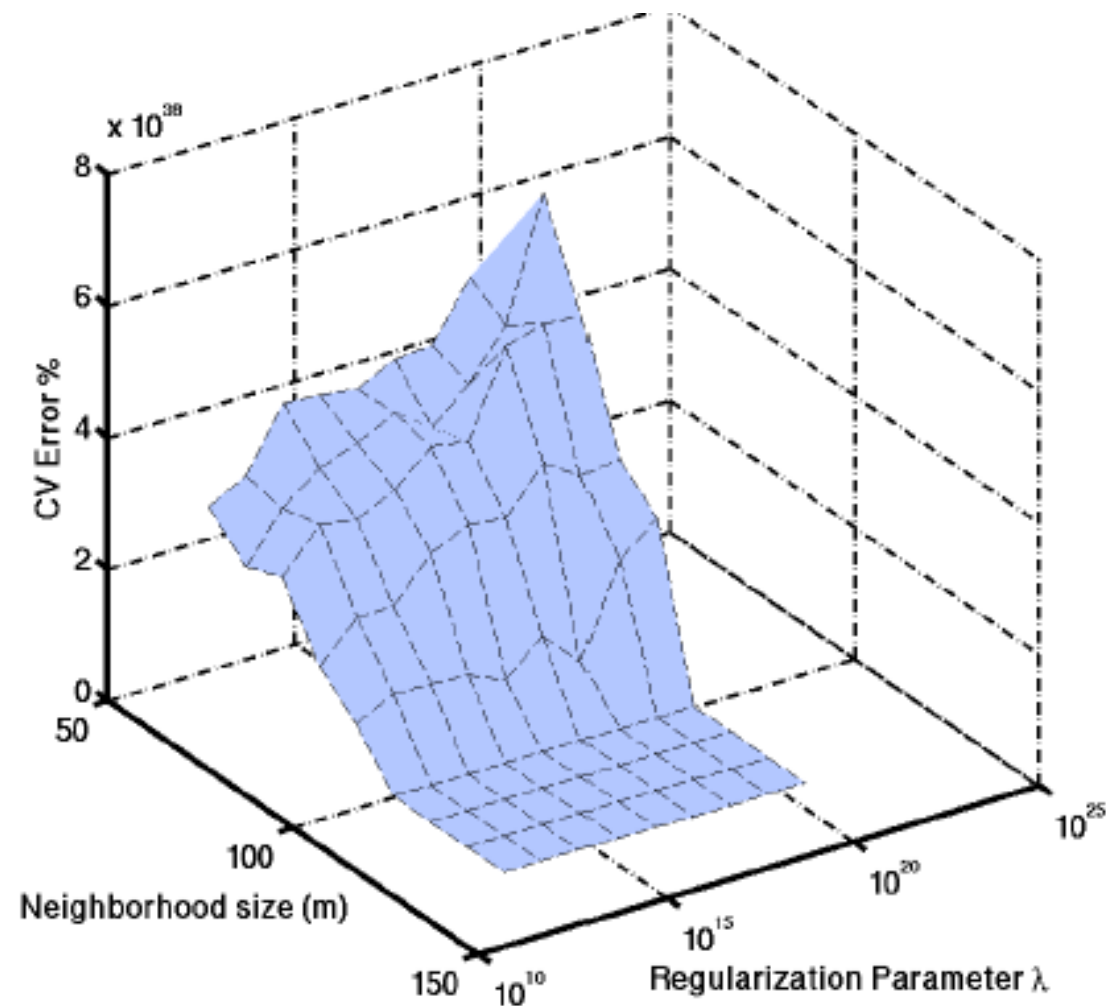
Example 2

Power Prediction at Fine-grain



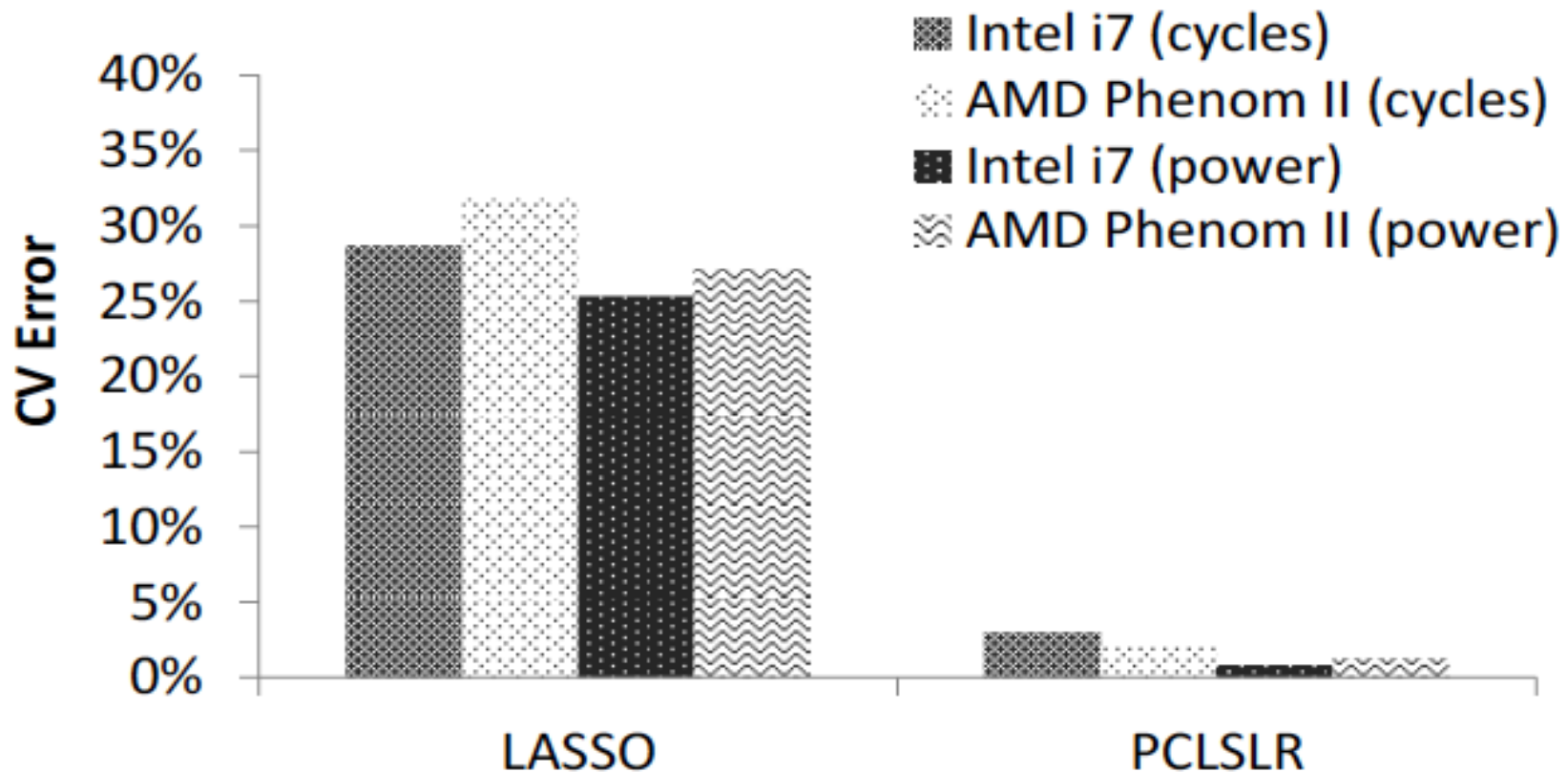
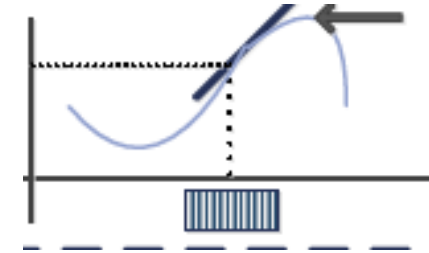
Example 2

Average Cross-Validation Error: 10-fold cross-validation

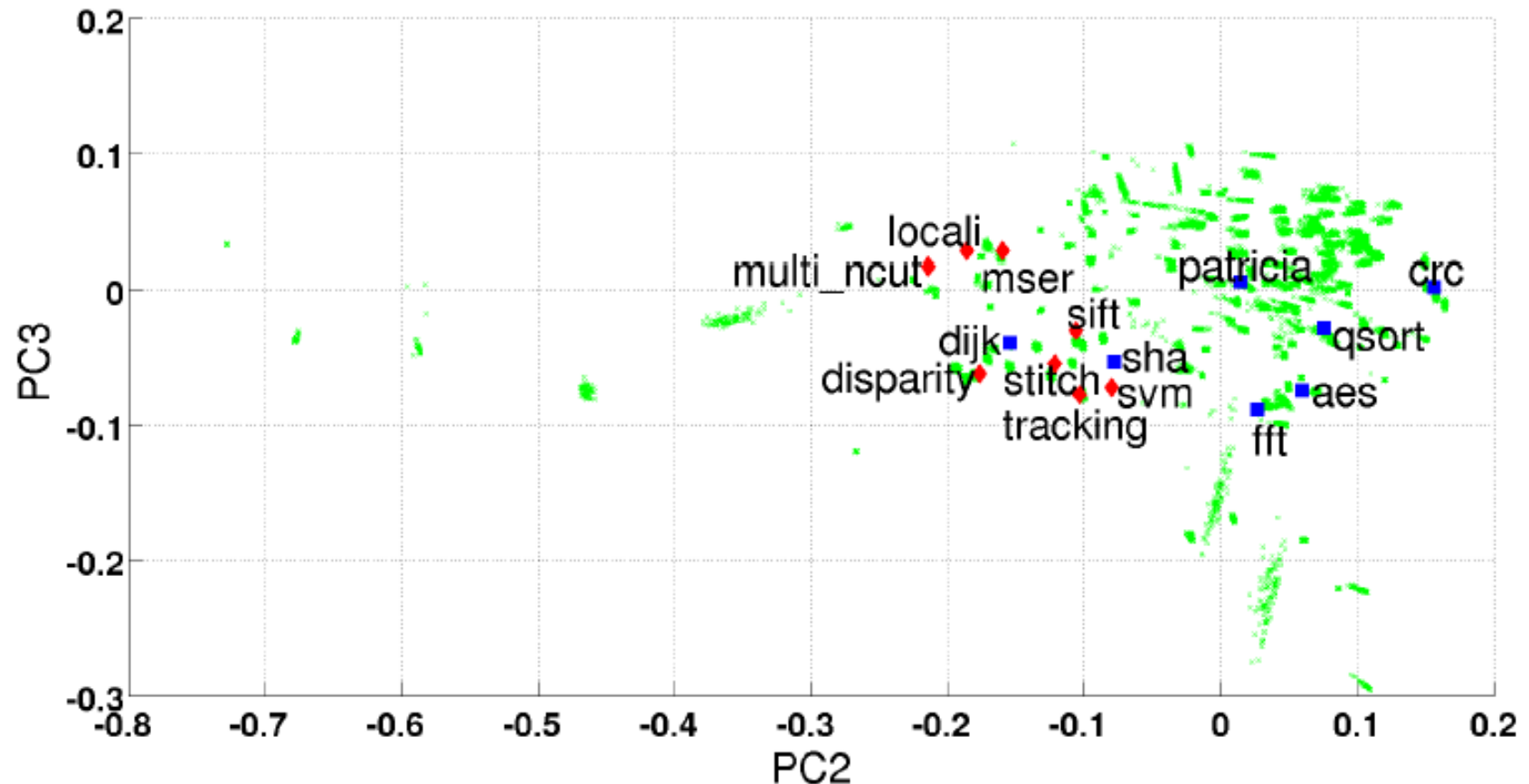


Example 2

Non-Linearity of F

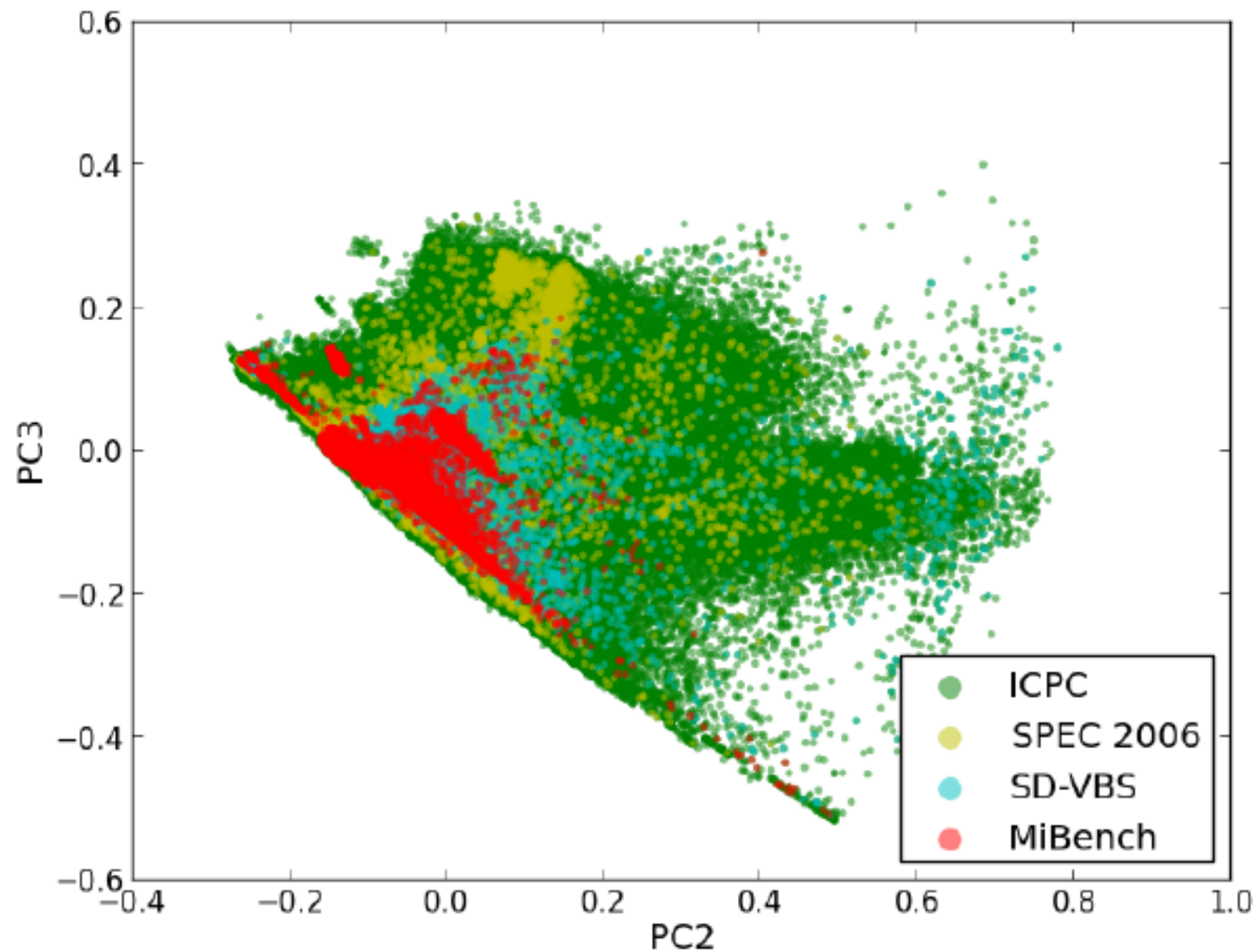


Example 2



Principal components (PC2 vs. PC3) of ICPC training applications (●), MiBench (■) and SD-VBS (◆) programs (Intel Core i7).

Example 2



Challenges in Cross-Platform Prediction

Host Sensitivity

Instrumenting Source Method

Aligning in No-Source (Perf Counter based) Methodology

Stochastic Dynamic Coupling

Requires Solving Regression during Prediction

More work to be done but cross-platform prediction seems feasible.

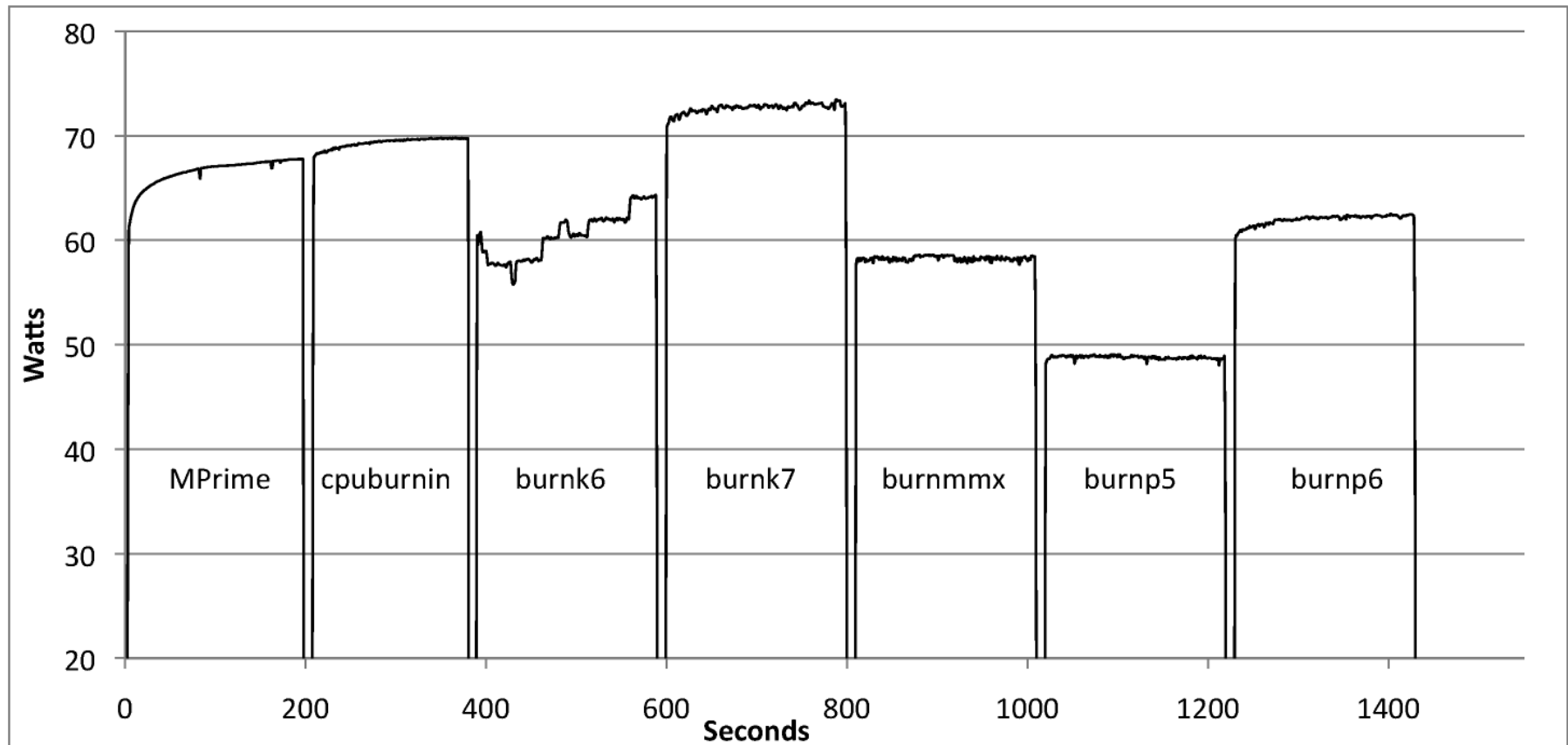
Challenges in Creating Max-power Viruses

- **Hand crafting code snippets for power viruses**
 - Very tedious process, complex interactions inside the processor
 - Cannot be sure if it is the maximum case
 - Heavily architecture dependent; heavy domain knowledge
- **Automatically generate power viruses**

Power Virus	Optimized to stress	Language
MPrime	All CPUs, all ISAs	C
CPUburn-in	X86 machines	x86 assembly
BurnP5	Intel Pentium w&w/o MMX processors	x86 assembly
BurnP6	Intel PentiumPro, Pentium II, Pentium III and Celeron CPUs	x86 assembly
BurnK6	AMD K6 processors	x86 assembly
BurnK7	AMD Athlon/Duron processors	x86 assembly
BurnMMX	cache/memory interfaces on all CPUs with MMX	x86 assembly

Example 3

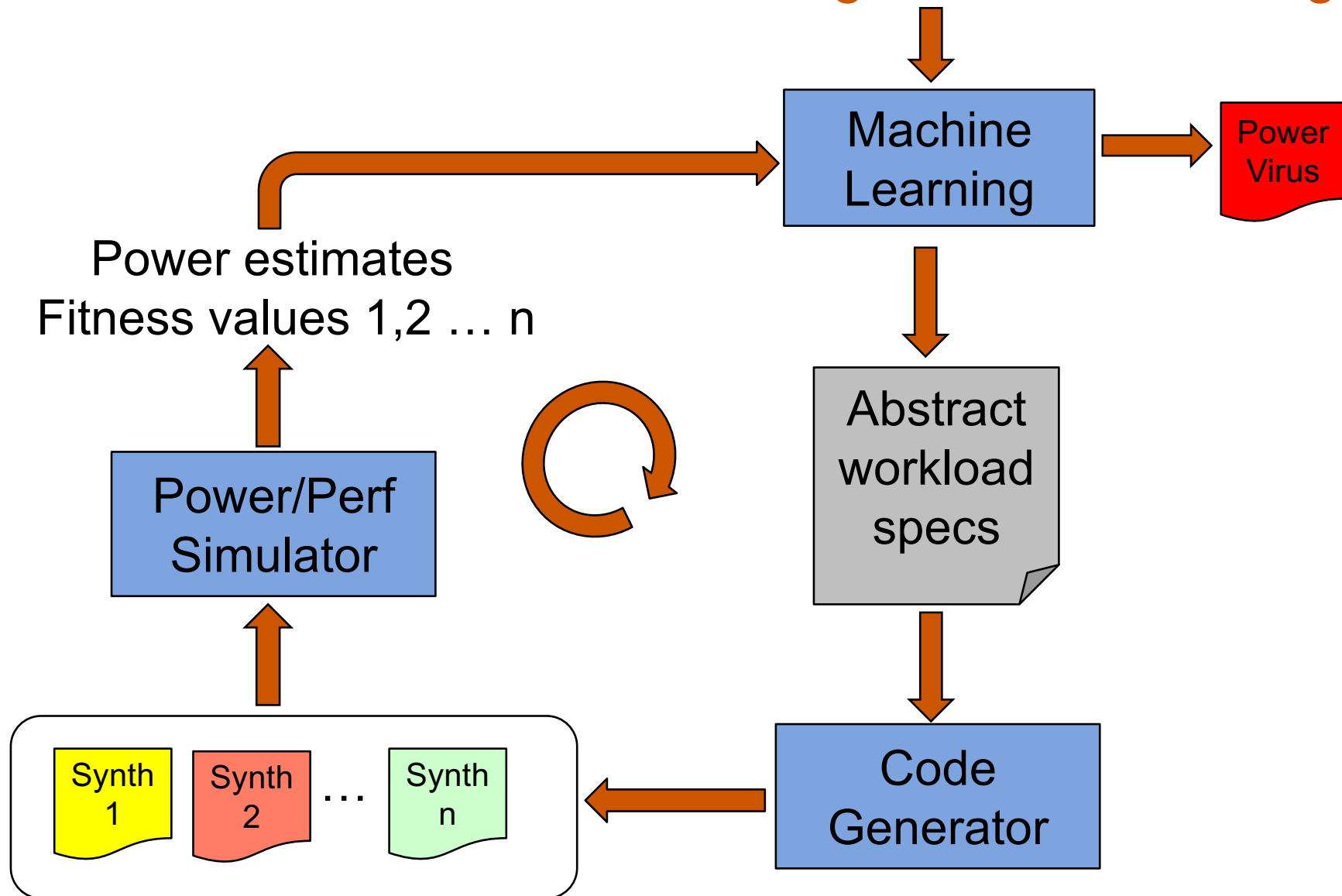
Power measurement of Viruses on Hardware



- **BurnK7 – 72.1 Watts**
- **SPEC CPU2006: 416.gamess and 453.povray consume highest power of 63.1 and 59.6 Watts**

Example 3

Power Proxies and Viruses using Machine Learning



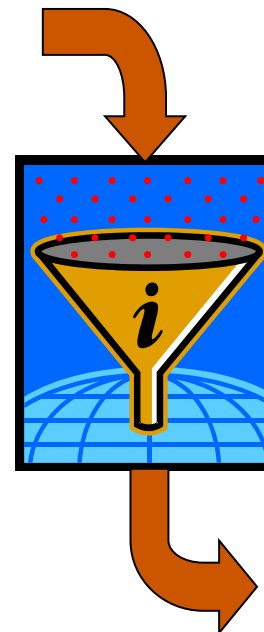
Proxy Workload Generation

- Derive proxy applications from a set of workload characterizations
- Proxies convey no proprietary information, but capture the execution behavior of developer's applications
- Proxy applications have similar power and performance characteristics as original

Original Workloads

Proxies are miniature and can be run on RTL

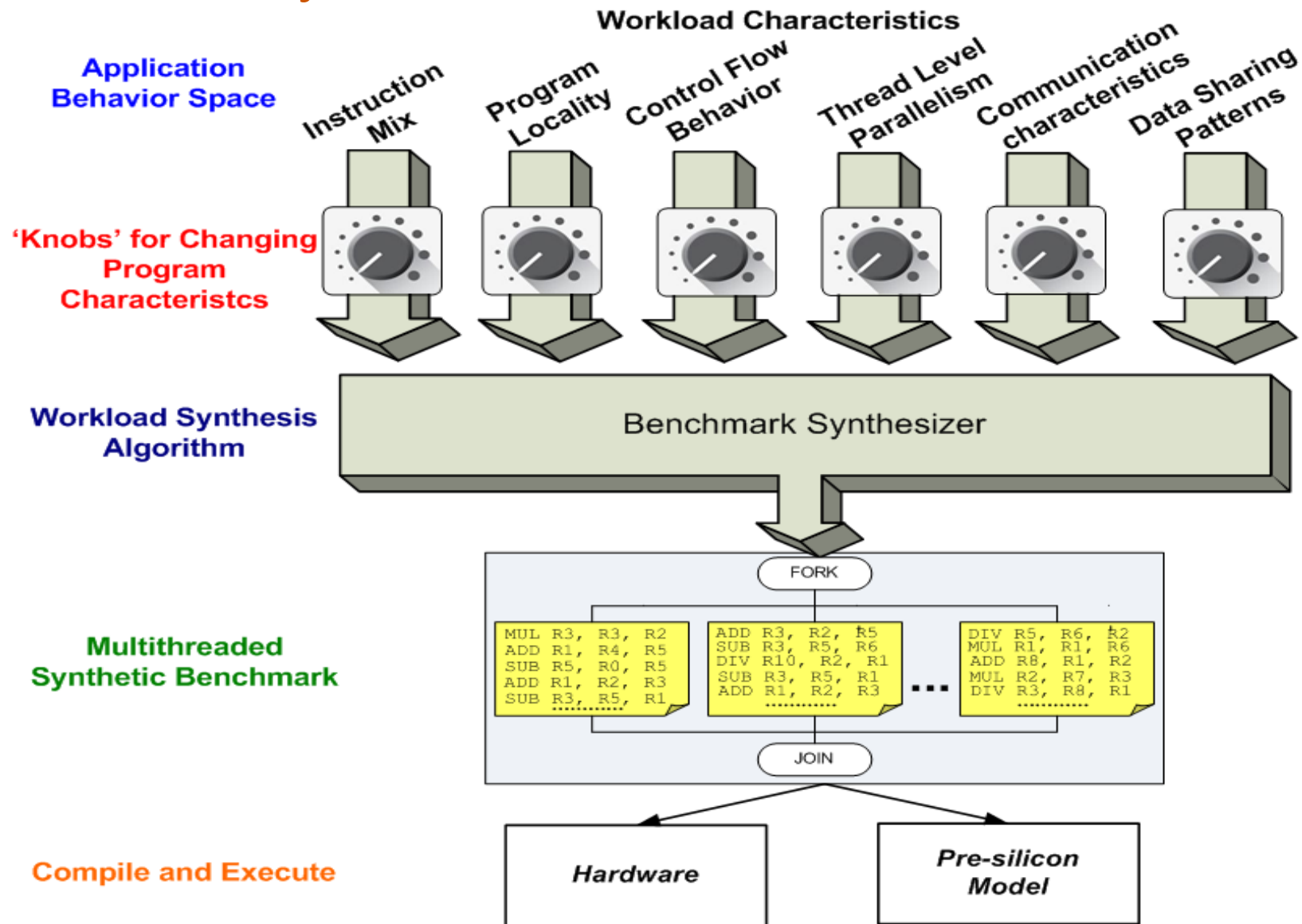
Power can be modeled on RTL without OS
and without software stack



Performance/Power Clones

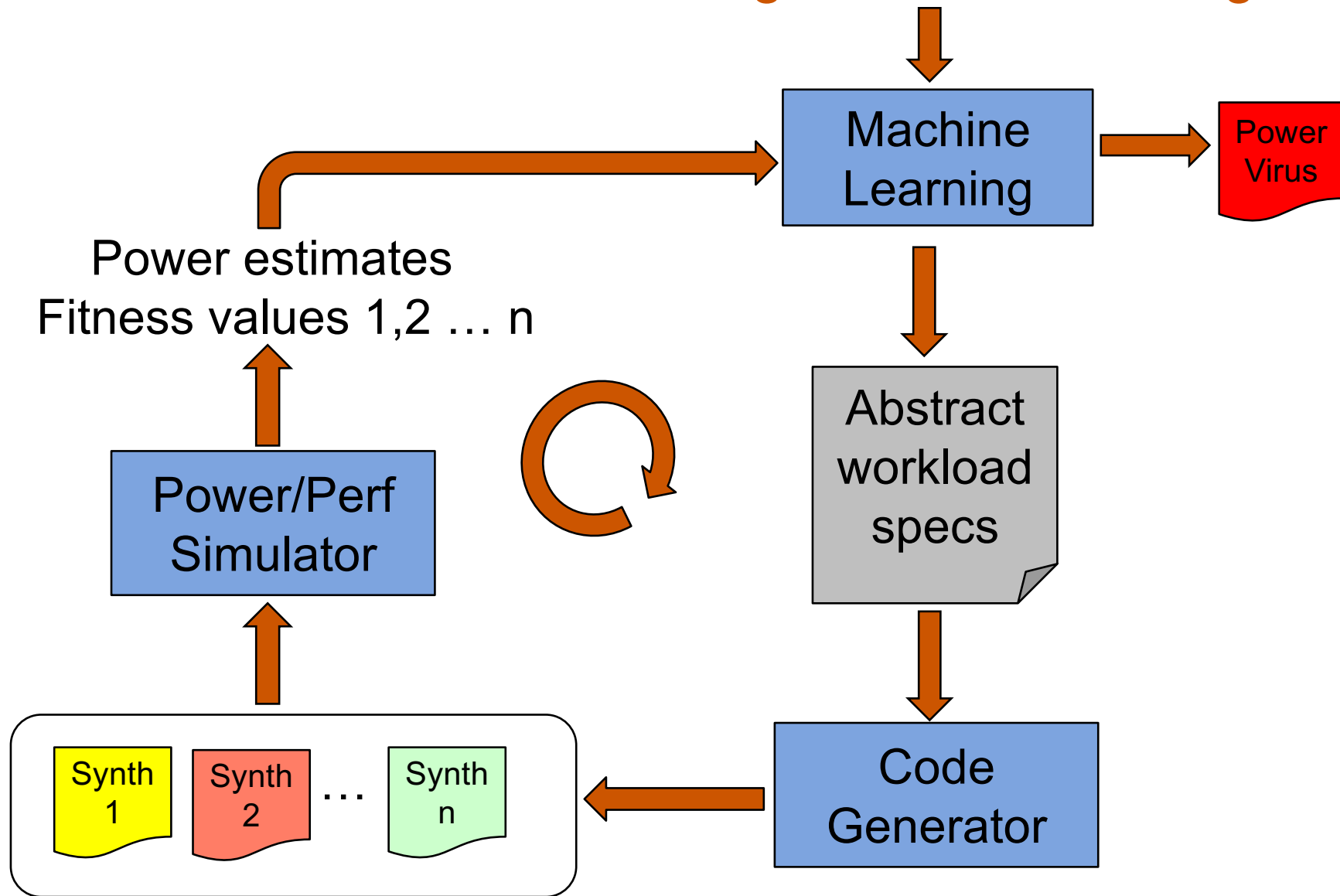
Example 3

Automatic Synthetic Benchmark Generation



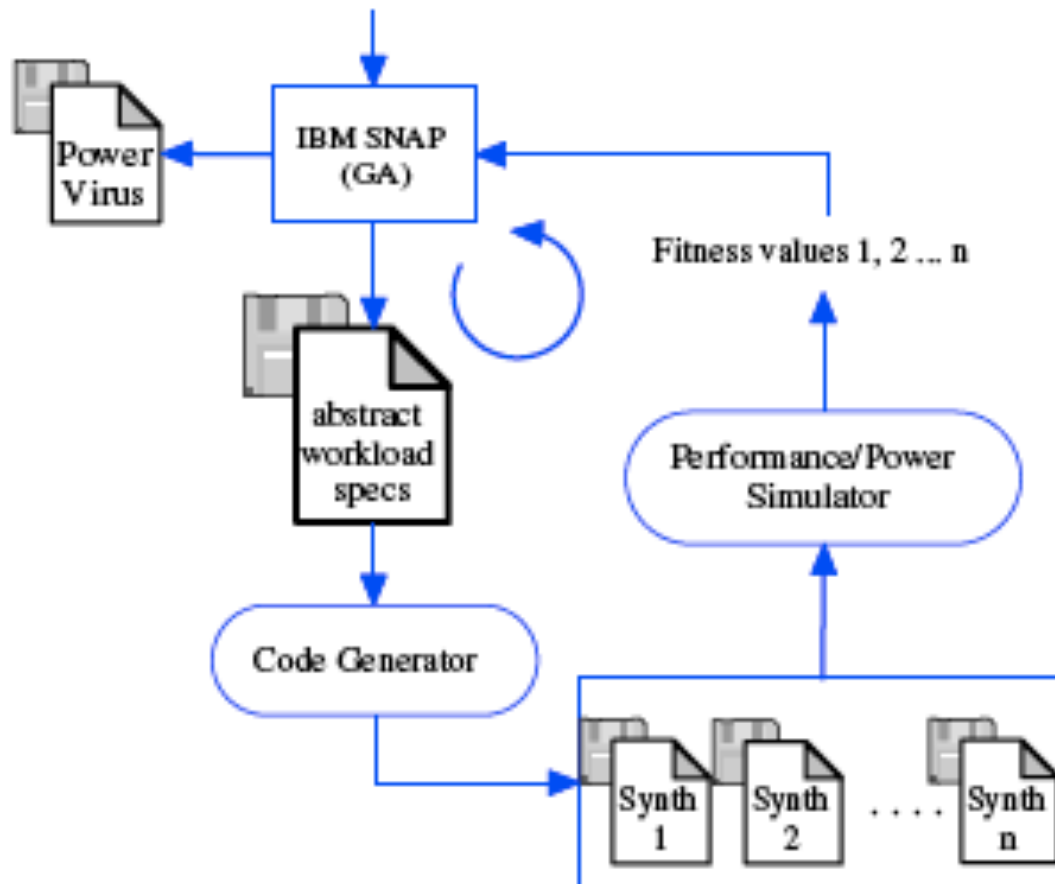
Example 3

Power Virus Generation using Machine Learning



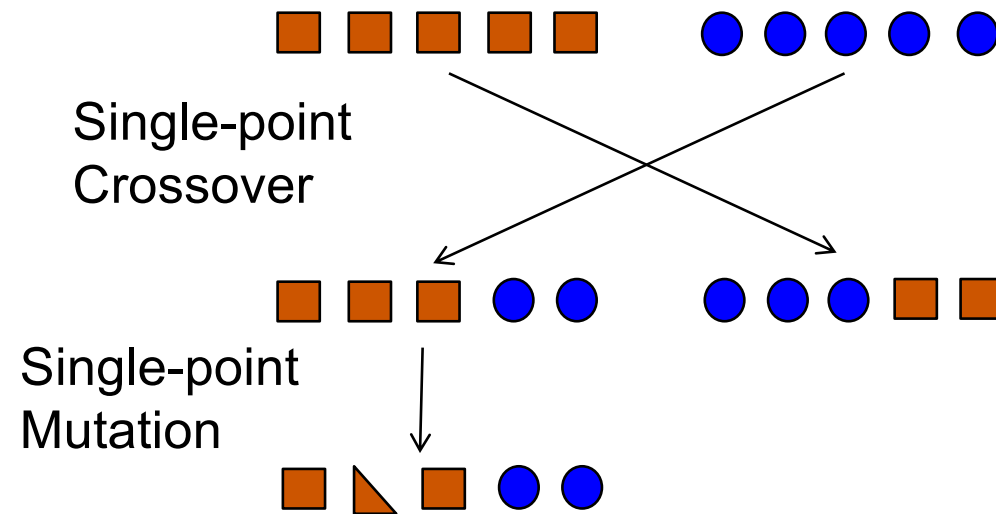
SYMPO and MAMPO Frameworks

- Automatically search for power viruses using an abstract workload model and machine learning



- GA: search heuristic to solve optimization problems
- Choose a random population, evaluate fitness, apply GA operators to generate next population
- Evolve until required fitness achieved

SYMPO Framework – Genetic Algorithm

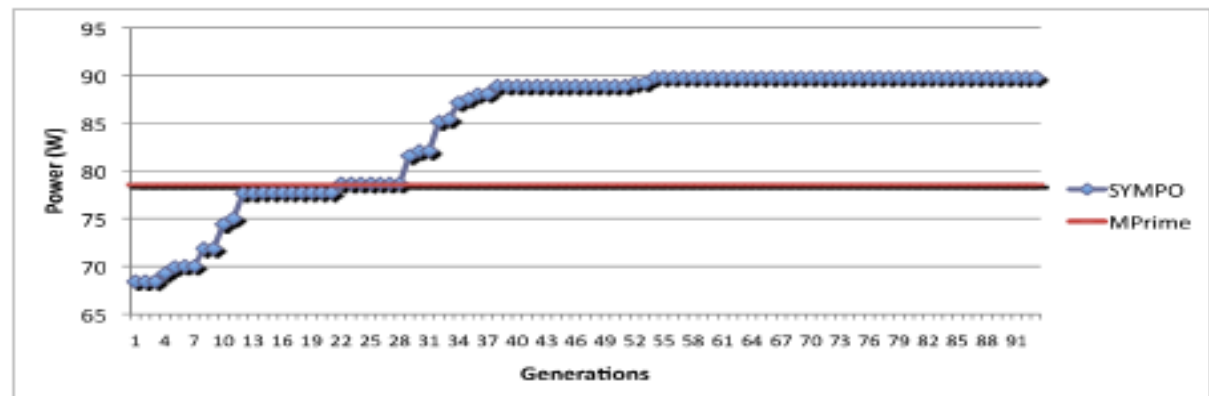


- Individuals -> synthetic workloads,
- Fitness function -> power on the design under study
- Mutation rate, reproduction rate, crossover rate

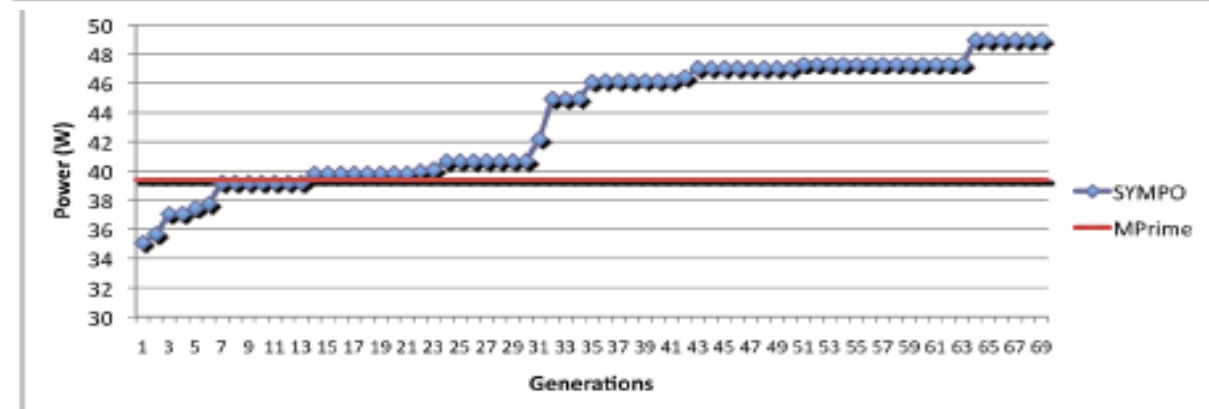
Example 3

SYMPO Vs Mprime on SPARC ISA

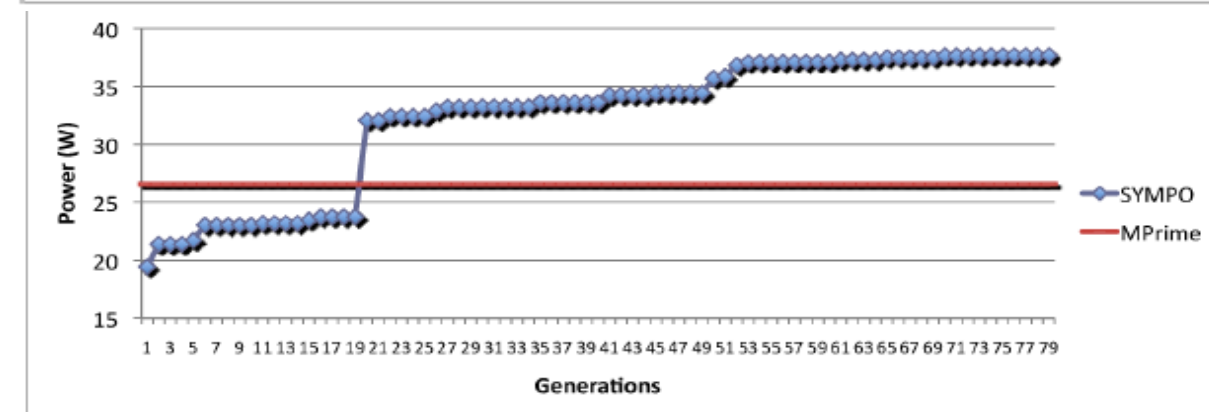
Config 1 - 14% more



Config 2 - 24% more

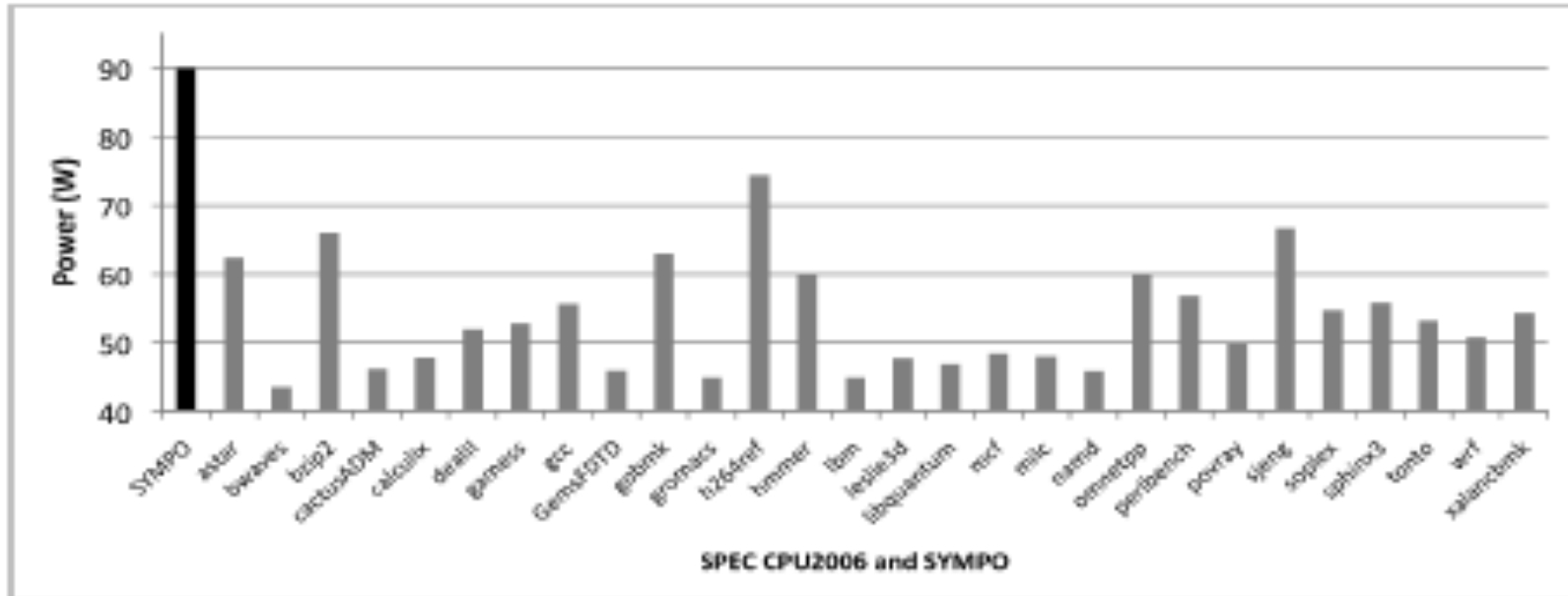


Config 3 - 41% more



Example 3

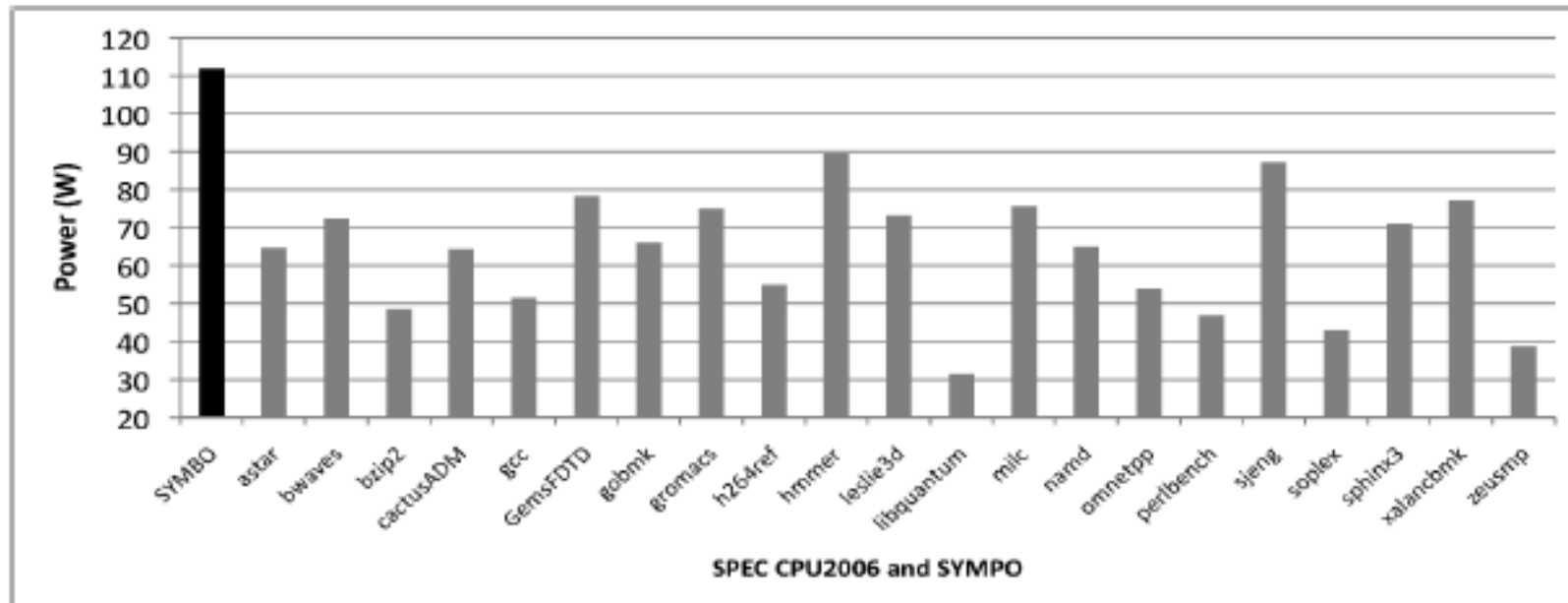
Comparison to SPEC CPU2006 on SPARC ISA



- **Comparison to SPEC CPU2006: 74.4Watts compared to 89.8Watts in SYMPO**

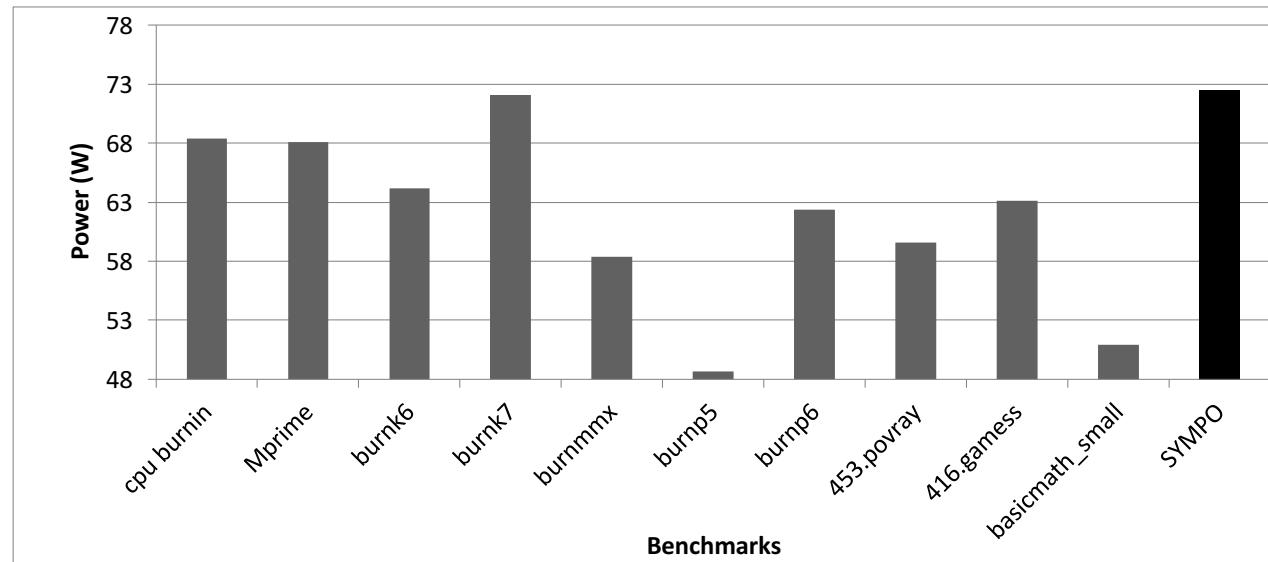
Example 3

Comparison to SPEC CPU2006 on Alpha ISA



- **Comparison to SPEC CPU2006: 111.8 Watts compared to 89.2 Watts, where theoretical maximum is 220 Watts**

Validation on x86 Hardware



- **The auto-generated stressmark (SYMPO) could beat the hand-tuned burnk7**

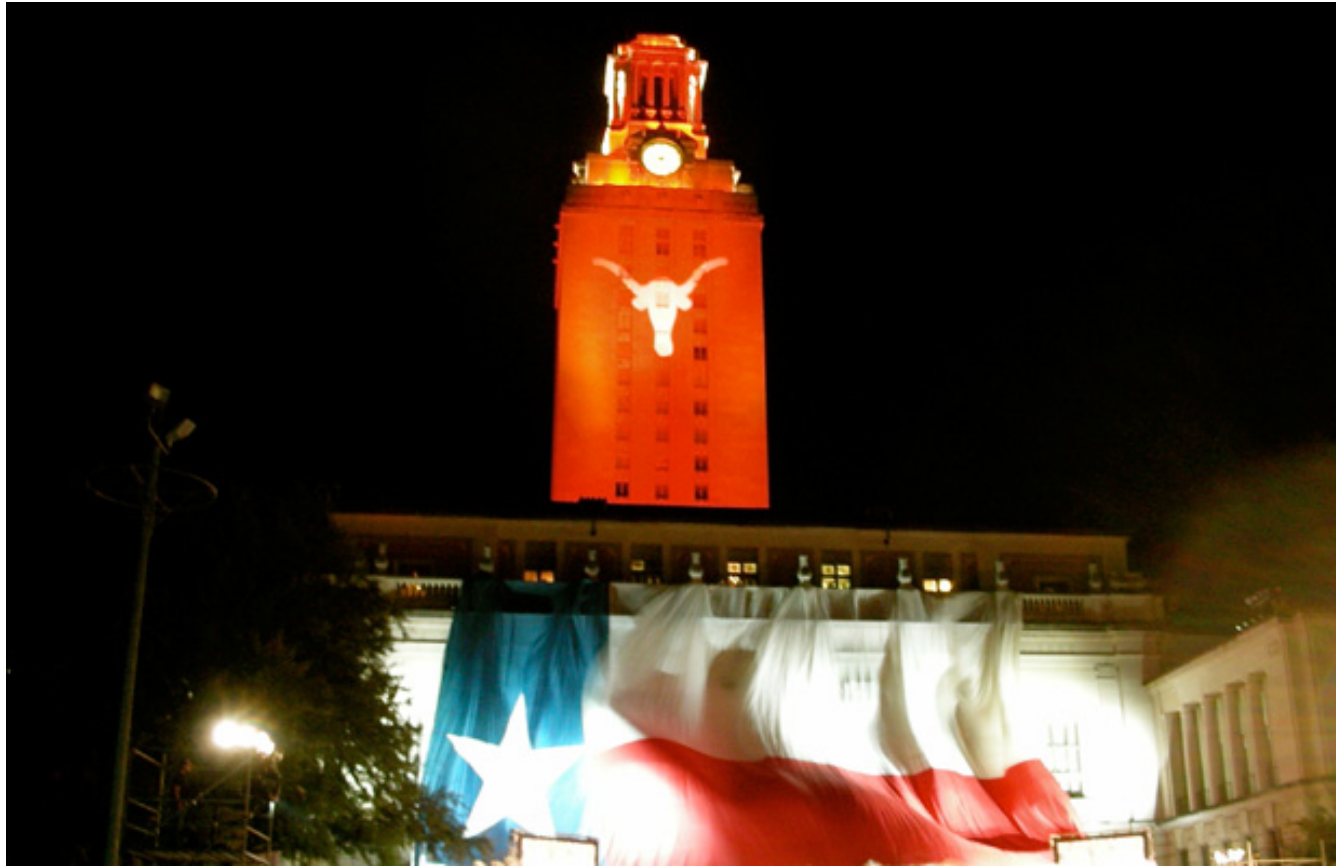
Summary

Machine Learning Techniques can be used to improve Power Modeling and Prediction.

Cross-Platform Prediction using Machine Learning can accurately track performance and power at phase level.

Synthetic Stressmarks created using Genetic Algorithms can excel hand-generated stressmarks.

Thank You! Questions?



Laboratory for Computer Architecture (LCA)
The University of Texas at Austin
lca.ece.utexas.edu